

Lab - Container Instances - Accessing Secrets within a Container

To Begin with the Lab

Summary

This lab demonstrates how to securely pass sensitive information (such as Azure Storage connection strings) to an Azure Container Instance (ACI) using **Secure Environment Variables**. The container reads the secret at runtime, accesses an Azure Storage Blob, and displays its contents without storing secrets inside the application code or Docker image.

Understanding

Applications often require sensitive information such as storage account keys, database passwords, API keys, or connection strings. Hardcoding these secrets into source code or Docker images is insecure because anyone with access to the image can retrieve them. Azure Container Instances provides secure methods for supplying secrets to containers at runtime:

- **Secure Environment Variables** – Secret values are securely injected into the container as environment variables.
- **Secret Volume Mounts** – Secrets are mounted as files inside the container.

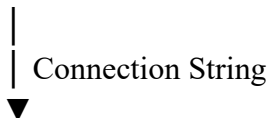
In this lab, the application retrieves the Azure Storage connection string from a secure environment variable. After receiving the connection string, it connects to Azure Blob Storage, reads the contents of a text file, and prints the data to the container logs.

Because this is a **console application**, it runs once, completes its task, and exits. Therefore, the container's **Restart Policy** is set to **Never**.

Example

Application Flow:

Azure Storage Account



Secure Environment Variable



Azure Container Instance



Console Application



Reads Blob File (data.txt)

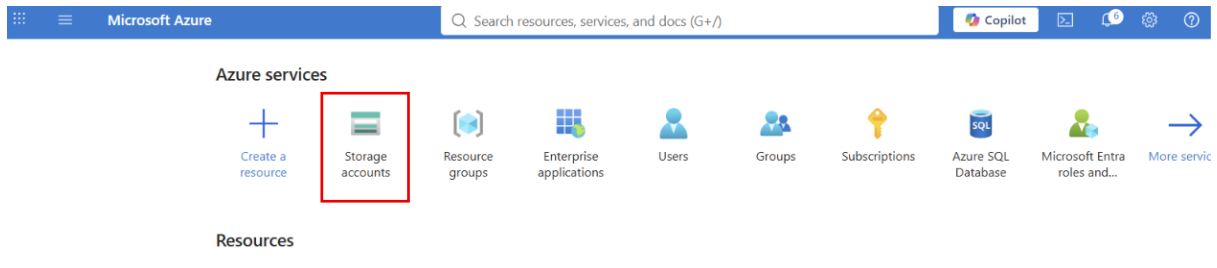


Displays Blob Contents in Container Logs

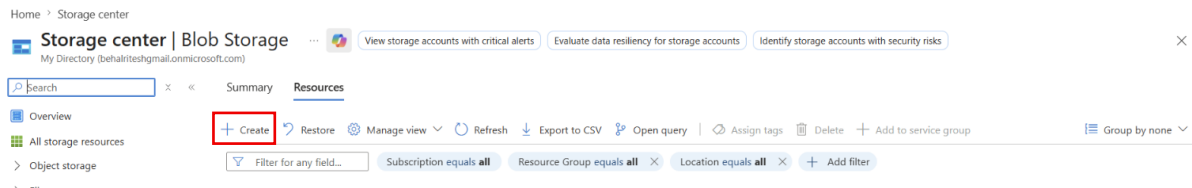
Lab Steps

Step 1: Create an Azure Storage Account

- Go to the **Storage accounts**.



- Create an **Azure Storage Account**.



- Subscription**.
- Resource Group** (create a new one if needed).
- Storage account name** (must be globally unique).
- Region** (US) East US.
- Performance**: Standard.
- Redundancy**: Locally redundant storage (or as required).
- Click **Review + Create**.
- Click **Create**.

Create a storage account

Project details

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription * Visual Studio Enterprise Subscription ▼

Resource group * Webapp ▼

[Create new](#)

Instance details

Storage account name * ⓘ mystdev34

Region * ⓘ (US) East US ▼

[Deploy to an Azure Extended Zone](#)

Primary service * Azure Blob Storage or Azure Data Lake Storage ▼

i This helps us provide relevant guidance. It doesn't restrict your storage to this resource type. [Learn more](#)

Performance * ⓘ

☒ **Standard:** Recommended for most scenarios (general-purpose v2 account)

☐ **Premium:** Recommended for scenarios that require low latency.

Redundancy * ⓘ Locally redundant storage (LRS) ▼

[Previous](#)[Next](#)[Review + create](#)

- Go to storage account just created.
- Navigate to Container.
- Create a **Data Container**.

Home > mystdev34

mystdev34 Storage account Containers ☆ ...

Search

Overview
Activity log
Tags
Diagnose and solve problems
Access Control (IAM)
Data migration
Events
Storage browser
Storage Mover
Partner solutions
Resource visualizer
Data storage
Containers

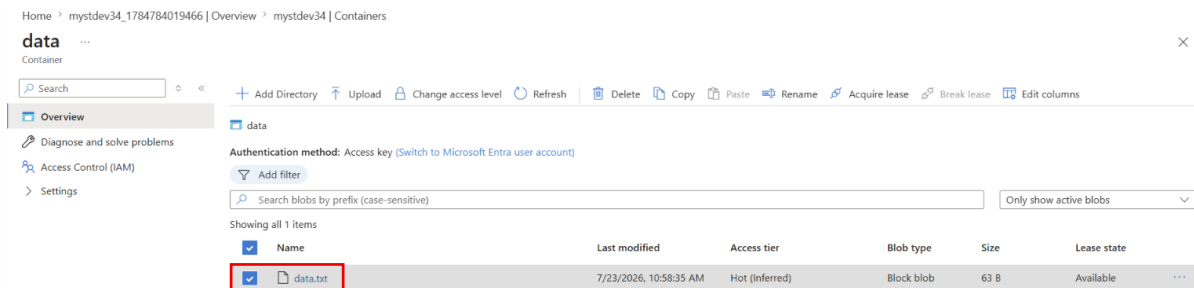
[+ Add container](#) [Upload](#) [Refresh](#) [Delete](#) [Change access level](#) [Restore containers](#) [Edit columns](#)

Search containers by prefix Only show active containers ▼

Showing all 2 items

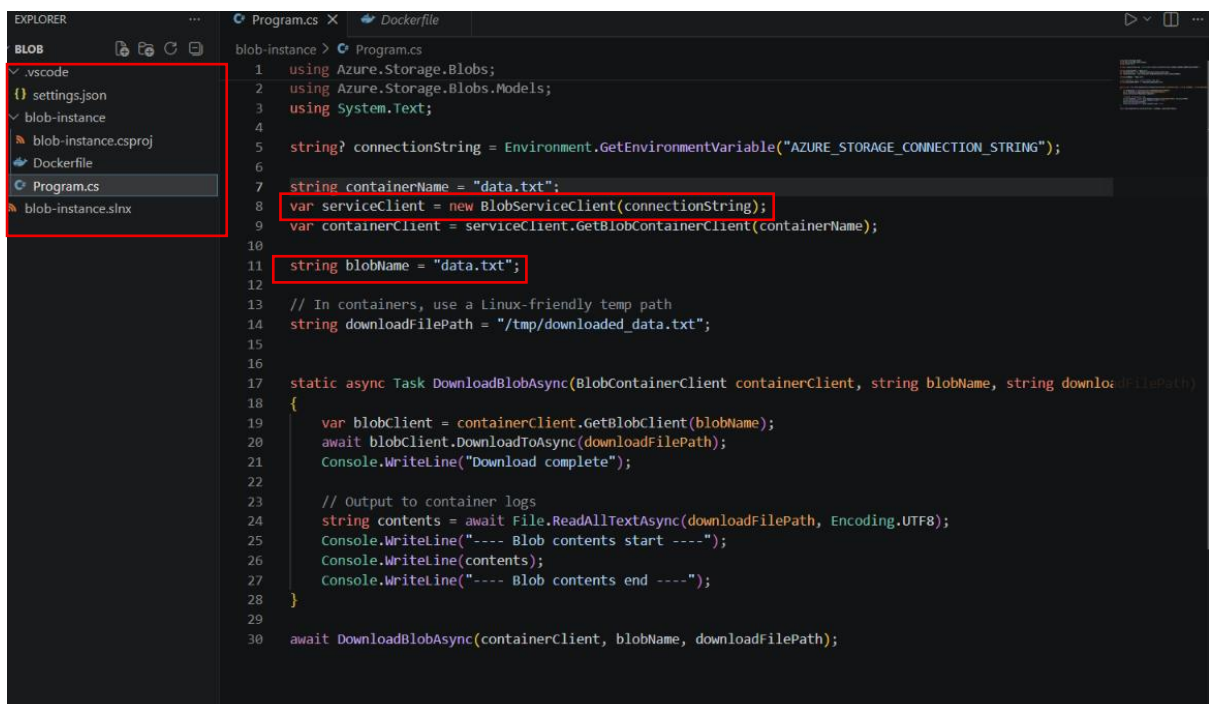
☐	Name	Last modified	Anonymous access level	Lease state	Storage connector	
☐	Blogs	7/23/2026, 10:51:10 AM	Private	Available	-	...

- Upload a text file (**data.txt**) into the container.
- Add sample text inside the file.



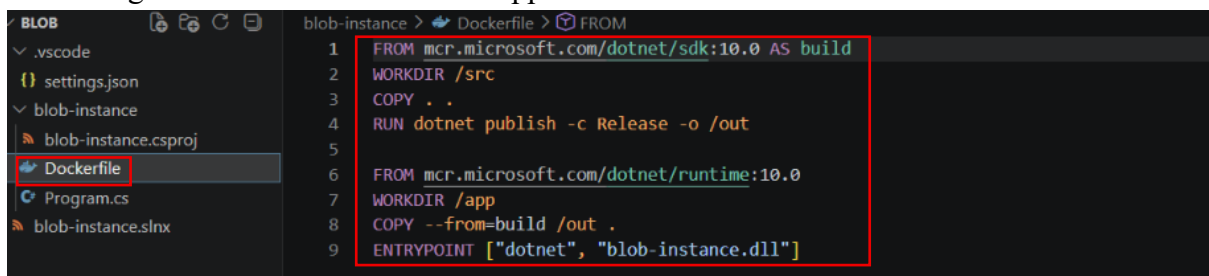
Step 2: Create the Console Application

- Develop a .NET console application.
- Read the storage connection string from an environment variable.
- Connect to Azure Blob Storage.
- Read the contents of the blob.



Step 3: Create the Dockerfile

- Create a multi-stage Dockerfile.
- Build the application.
- Publish the application.
- Configure the container to run the application DLL.



Step 4: Build the Docker Image

- Open the terminal.
- Navigate to the project directory.

- Build the Docker image.

```
docker build -t blobapp:v1 .
```

```
PS D:\SIMPLE APP\Blob> cd .\blob-instance
PS D:\SIMPLE APP\Blob\blob-instance> docker build -t blobapp:v1 .
[+] Building 18.8s (13/13) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 279B
=> [internal] load metadata for mcr.microsoft.com/dotnet/sdk:10.0
=> [internal] load metadata for mcr.microsoft.com/dotnet/runtime:10.0
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [build 1/4] FROM mcr.microsoft.com/dotnet/sdk:10.0@sha256:ed034a8bf0b24ded0cbbac07e17825d8e9ebfe21e308191d0f7421eaf5ad4664
=> => resolve mcr.microsoft.com/dotnet/sdk:10.0@sha256:ed034a8bf0b24ded0cbbac07e17825d8e9ebfe21e308191d0f7421eaf5ad4664
=> [internal] load build context
=> => transferring context: 1.91kB
=> [stage-1 1/3] FROM mcr.microsoft.com/dotnet/runtime:10.0@sha256:ed5d539b27842d656a06a5984dbcb5114d3e885fbada612a49a5a7c3c3a44e1c
=> => resolve mcr.microsoft.com/dotnet/runtime:10.0@sha256:ed5d539b27842d656a06a5984dbcb5114d3e885fbada612a49a5a7c3c3a44e1c
=> CACHED [build 2/4] WORKDIR /src
=> [stage-1 2/3] WORKDIR /app
=> [build 3/4] COPY . .
=> [build 4/4] RUN dotnet publish -c Release -o /out
=> [stage-1 3/3] COPY --from=build /out .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:6a5f55a34c2af9f12d805795f768f35fe5fabfc94deef2b4e5f7bed4d384eb57
=> => exporting config sha256:d120ec9bc86afca81086f5ef9adb989e599ed93bb1b916245b0338b944001a9d
=> => exporting attestation manifest sha256:5c877e3c0e2d4c85e7f80c1219665ec99aac1db17169ef7a1e8ec05ffde6c8f0
=> => exporting manifest list sha256:643f334aa49dbc9f616e0ac4785e5efbcacbc8c4e5ae855b35b7abab7660833
=> naming to docker.io/library/blobapp:v1
=> unpacking to docker.io/library/blobapp:v1
PS D:\SIMPLE APP\Blob\blob-instance>
```

Step 5: Tag the Docker Image

- Go to the container registry.
- Copy the Login server.



- Tag the local Docker image.
- Associate it with the Azure Container Registry repository.
- Paste inside the command.

```
docker tag blobapp:v1 regdeves12.azurecr.io/blobapp:v1
```

```
PS D:\SIMPLE APP\Blob\blob-instance> docker tag blobapp:v1 regdeves12.azurecr.io/blobapp:v1
PS D:\SIMPLE APP\Blob\blob-instance>
```

Step 6: Push the Image to Azure Container Registry

- Log in to Azure Container Registry.
- Push the image to the registry.
- Paste the login server inside this command.

```
docker push regdeves12.azurecr.io/blobapp:v1
```

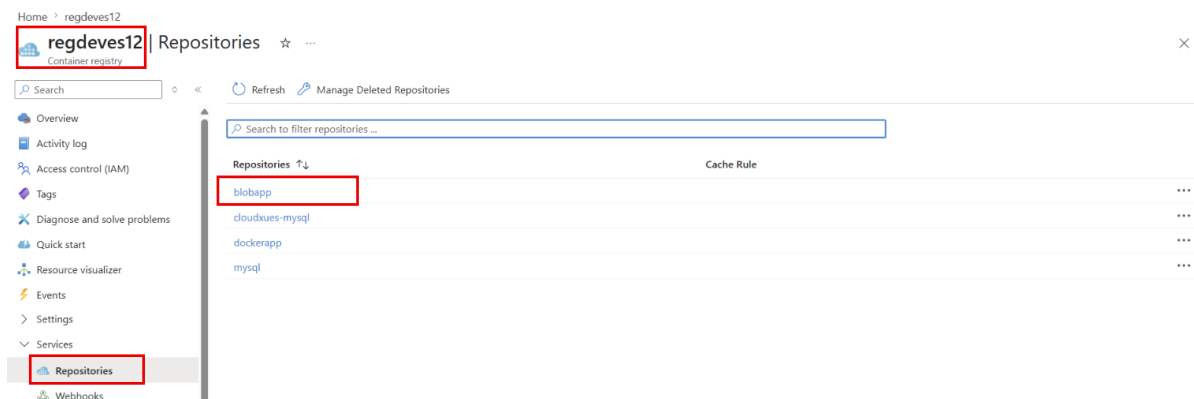
```

PS D:\SIMPLE APP\Blob\blob-instance> docker tag blobapp:v1 regdeves12.azurecr.io/blobapp:v1
PS D:\SIMPLE APP\Blob\blob-instance> docker push regdeves12.azurecr.io/blobapp:v1
The push refers to repository [regdeves12.azurecr.io/blobapp]
ca2678b2b780: Mounted from dockerapp
cdf564afb68f: Mounted from dockerapp
7990facc64df: Mounted from dockerapp
8e782b78af5d: Mounted from dockerapp
65831f9a448c: Pushed
0287f08f0ac0: Mounted from dockerapp
a7e36c4b28c5: Pushed
5e727aa3601f: Pushed
v1: digest: sha256:643f334aa49dbc9fe16e0ac4785e5efbcacbc8c4e5ae8555b35b7abab7660833 size: 856
PS D:\SIMPLE APP\Blob\blob-instance>

```

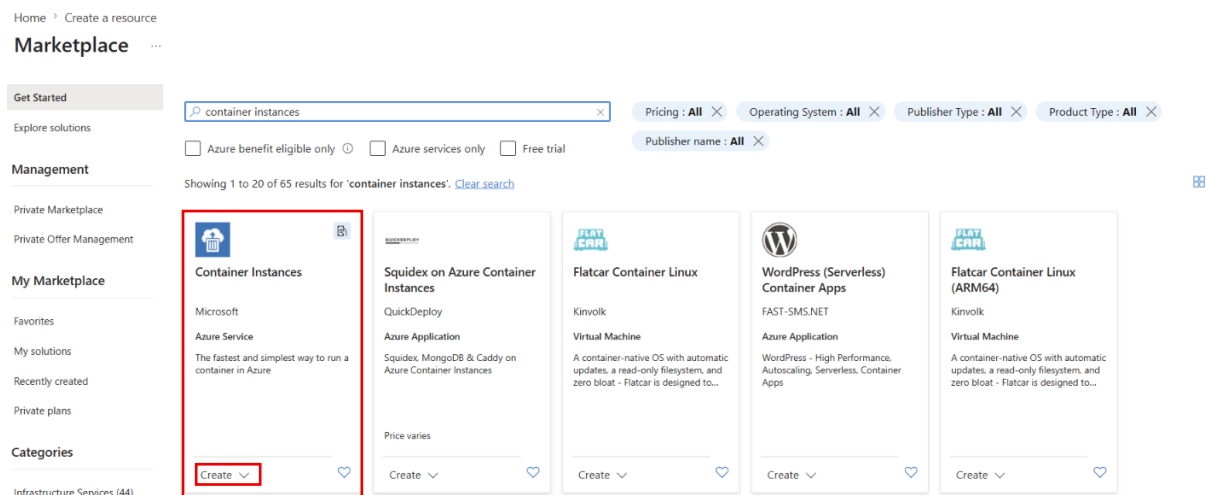
Step 7: Verify the Image

- Open Azure Portal.
- Navigate to **Azure Container Registry**.
- Open **Repositories**.
- Confirm that the **blobapp:v1** image exists.



Step 8: Create an Azure Container Instance

- Open Azure Portal.
- Create a new **Azure Container Instance**.



- Select the **Resource Group**.
- Enter the any **container name**.
- Select the **deployment region**.
- Choose the **Azure Container Registry**.
- Select the **blobapp:v1** image.

Create container instance ...

Subscription * ⓘ Visual Studio Enterprise Subscription

Resource group * ⓘ Webapp [Create new](#)

Container details

Container name * ⓘ blobapp ✓

Region * ⓘ (US) East US

Availability zones (Preview) ⓘ None

SKU Standard

Image source * ⓘ ☐ Quickstart images ☒ Azure Container Registry ☐ Other registry

Run with Azure Spot discount ⓘ ☐
 ⓘ Spot containers are not available in the selected region. [Learn more](#)

Registry * ⓘ regdevs12
 ⓘ If you do not see your Azure Container Registry, ensure you have been assigned the Reader Role for the Azure Container Registry or select an Azure Container Registry in a different subscription. [Learn more](#)

Image * ⓘ blobapp

Image tag * ⓘ v1

OS type * ☒ Linux ☐ Windows

Step 9: Configure Networking

- Open the **Networking** section.
- Select **None** because public access is not required.

Microsoft Azure Search resources, services, and docs (G+/I)

Home > Create a resource > Marketplace

Create container instance ...

Basics **Networking** Monitoring Advanced Tags Review + create

Choose between three networking options for your container instance:

- **'Public'** will create a public IP address for your container instance.
- **'Private'** will allow you to choose a new or existing virtual network for your container instance.
- **'None'** will not create either a public IP or virtual network. You will still be able to access your container logs using the command line.

Networking type ☐ Public ☐ Private ☒ None

Step 10: Configure Secure Environment Variable

- Open the **Advanced** section.
- Add a new Environment Variable.
- Enter the variable name.
- Mark it as **Secure (Yes)**.
- Copy the Azure Storage connection string from the Storage Account.
- Paste it as the secure value.

Variable Name: **AZURE_STORAGE_CONNECTION_STRING**

- Go to the Storage account.
- Navigate to Access keys.
- Copy the **Connection string**.

Home > mystdev34



mystdev34 | Access keys ☆ ...

Storage account

Search

Overview

Activity log

Tags

Diagnose and solve problems

Access Control (IAM)

Data migration

Events

Storage browser

Storage Mover

Partner solutions

Resource visualizer

Data storage

Security + networking

Networking

Front Door and CDN

Access keys

Set rotation reminder Refresh Give feedback

Access keys authenticate your applications' requests to this storage account. Keep your keys in a secure location like Azure Key Vault, and replace them often with new keys. The two keys allow you to replace one while still using the other.

Remember to update the keys with any Azure resources and apps that use this storage account.

[Learn more about managing storage account access keys](#)

Storage account name

mystdev34

key1 Rotate key

Last rotated: 7/23/2026 (0 days ago)

Key

.....

Show

Connection string

DefaultEndpointsProtocol=https;AccountName=mystdev34;AccountKey=15L3gU...

Hide

key2 Rotate key

Last rotated: 7/23/2026 (0 days ago)

Key

.....

Show

Connection string

- Paste it as the secure value.

Value: <Storage Connection String>

Home > Create a resource > Marketplace

Create container instance ...

Basics Networking Monitoring **Advanced** Tags Review + create

Configure additional container properties and variables.

Restart policy ⓘ

Never

Environment variables

Mark as secure	Key	Value
Yes	AZURE_STORAGE_CONNECTIO...	
No		

Command override ⓘ

[]

Example: ["/bin/bash", "-c", "echo hello; sleep 100000"]

Key management ⓘ

☒ Microsoft-managed keys (MMK)

☐ Customer-managed keys (CMK)

Step 11: Configure Restart Policy

- Set **Restart Policy** to **Never**.
- Since the application is a console application, it only needs to execute once.

Create container instance ...

Basics Networking Monitoring **Advanced** Tags Review + create

Configure additional container properties and variables.

Restart policy ⓘ	Never	▼
------------------	-------	---

Environment variables

Mark as secure	Key	Value
Yes ▼	AZURE_STORAGE_CONNECTIO... ✓	 ✓
No ▼		

Command override ⓘ Example: ["/bin/bash", "-c", "echo hello; sleep 100000"]

Key management ⓘ ☒ Microsoft-managed keys (MMK) ☐ Customer-managed keys (CMK)

Step 12: Deploy the Container

- Click **Review + Create**.
- Validate the configuration.
- Create the Azure Container Instance.

Create container instance ...

✓ Validation passed

Basics Networking Monitoring Advanced Tags **Review + create**

Basics

Subscription	Visual Studio Enterprise Subscription
Resource group	Webapp
Region	East US
Container name	blobapp
SKU	Standard
Image type	Private
Image registry login server	regdeves12.azurecr.io
Image	regdeves12.azurecr.io/blobapp:v1
Image registry user name	regdeves12
OS type	Linux
Memory (GiB)	1.5
Number of CPU cores	1
GPU type (preview)	None
GPU count	0

Networking

Networking type	None
-----------------	------

Create

< Previous

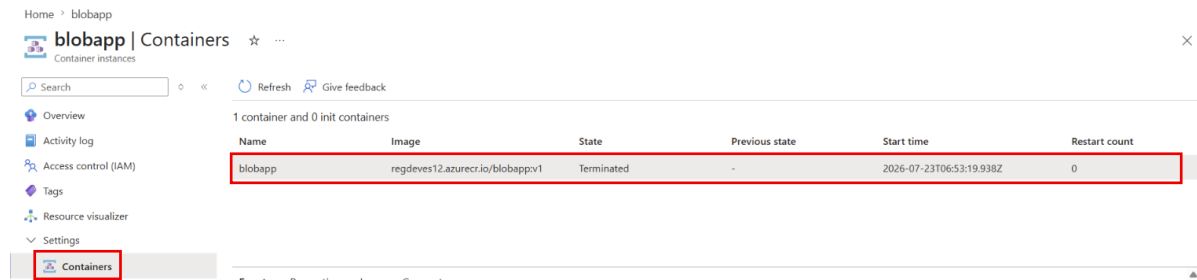
Next >

[Download a template for automation](#)

Step 13: Verify Container Status

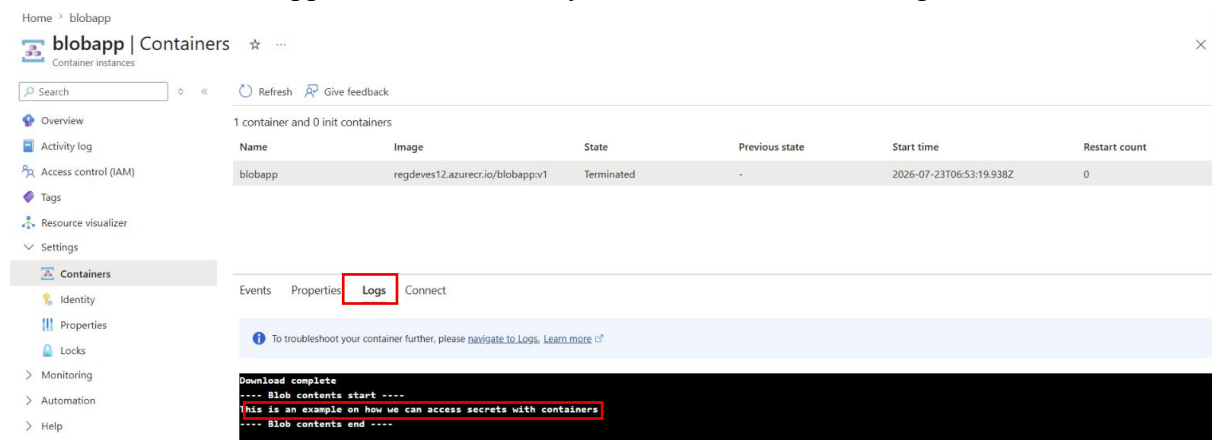
- Open the deployed Container Instance.
- Navigate to **Containers**.

- Verify that the container has completed execution.
- The status should be **Terminated**.



Step 14: View Container Logs

- Open the **Logs** section.
- Verify that the contents of **data.txt** are displayed.
- Confirm that the application successfully connected to Azure Storage.



Step 15: Understand Azure File Share Mounts

- Azure Container Instances also support Azure File Share mounting.
- Mount Azure File Shares as persistent storage.
- Data stored in Azure File Shares remains available even after the container stops.
- Use Azure File Shares whenever persistent storage is required.

Important Commands

Build Docker Image

`docker build -t blobapp:v1 .`

Tag Docker Image

`docker tag blobapp:v1 <acr-name>.azurecr.io/blobapp:v1`

Push Docker Image

`docker push <acr-name>.azurecr.io/blobapp:v1`